Automated Local Data Pipeline & Data Apps

v1 by Dennis Tam

VS Code + GitHub Copilot vs v0.dev

VS Code + GitHub Copilot

Full development environment

- Web, Python, MQL4 for MetaTrader, Swift for iOS/macOS, and more.
- Existing codebases, backend logic, integrations, testing, and deployment.
- Direct control over architecture and implementation.

VS

v0.dev

Web UI and app generator

- Mainly for web applications, especially UI-first prototypes and landing pages.
- Quickly generates and iterates on front-end interfaces from prompts.
- Not a general-purpose IDE for Python, MQL4, Swift, or other non-web development.



TAKEAWAY:

Use VS Code + GitHub Copilot for broad, end-to-end development.
Use v0.dev when you want to quickly create a web interface.

VS Code + GitHub Copilot

Best for: projects needing deep control, custom logic, and long-term maintainability.

- Best when working in an existing codebase or building a complex product.
- Direct control over architecture, integrations, testing, and deployment.
- Flexible as requirements evolve, with room for detailed changes.

What you will learn

- Use build.md template to create local data apps with Python
- Integrate LLM multi-modal capability to process files

Business Case

- We are an HR department, we have received a number of CV for a particular job-posting
- We want to use LLM to help screen candidates against a predetermined scoring template
- Per generation, we want to generate an interview bundle, which contains the following
  - A folder of selected candidates CV
  - A scoring sheet for the screened candidates
  - A sheet of interview questions for each candidates

Functional Requiremnet

- We want a local python app, simple, modern and friendly interface
- We want to use file folder as data storage and structure, not complicated data-base
- We want to have multiple folder to handle entity
  - /candidates/batch_xx (handle groups of CV in .pdf)
  - /listing/listing_xx/(handles group of job listing, each sub containing listing_xx.txt + one scoring template.txt)
  - /output/timestamp_202609291129/ (handle each batch generation)
- We will use Openrouter (deepseek/deepseek-v4-flash) as the default model, in the interface, allow user to enter key and model handle directly
- Setup virtual environment to execute the project

Local HR Screening App: Architecture

A single-user local workflow with file-based storage and a controlled LLM boundary

1. LOCAL BROWSER UI STREAMLIT

Folder Selection
Pick candidates, listing & settings

Validation Review

Settings
Model handling, limits

Progress
Real-time status and steps

Open generated bundle folder

2. APPLICATION SERVICES (LOCAL)

Input Discovery	Discover CV PDFs in candidates and job listing in listing
Validation	Validate files, settings and environment (Pydantic models)
PDF Text Extraction (pydfff)	Extract text from CV PDFs (ppdf)
Candidate Screening Orchestration	Prepare payloads, batching and call OpenRouter API
Ranking & Selection	Score candidates, rank and select top N
Interview Question Generation	Generate tailored interview questions for selected candidates
Markdown Bundle Writer	Write timestamps bundle (ranking, summaries, questions) to (output)

3. EXTERNAL LLM BOUNDARY

OpenRouter API

OpenRouter API

Configurable Model Handle
(e.g., opera/uptext-40-mini)

Receives ONLY:

- ✔ Selected job listing text (listing)
- ✔ Scoring guidance (from settings)
- ✔ Extracted CV text (per candidate)

No files, paths, system prompts or API keys are sent.

4. LOCAL FILE-SYSTEM STORAGE

- **/candidates** PDF CVs
- **/listing** Job listing text/MD
- **/settings** Local settings (excluded from Git)
- **/temp** Temporary work (auto-cleaned)
- **/output** TimESTAMPed results bundles (MD)

5. SECURITY & RELIABILITY

- **API key never in source, GIF, reports or logs**
- **No database or login** (single-user)
- **Fail-fast** on error with cleanup
- **No partial completed bundle**

Python 3.11+ Streamlit

Local HR Candidate Screening: Data Flow

From folders and CVs to ranked candidates and interview bundles



1 HR prepares input folders

/candidates/ batch_xx

PDF CVs

/listing/ listing_xx

listing_xx.txt

scoring template.txt

2 HR selects inputs and settings

Candidate batch (batch, xx)

Job listing (listaxx.txt)

Top-candidate count

API key

Model (LLM)

3 Streamlit validates & extracts text from every PDF

PDF

Extract text from every PDF using pypdf

4 OpenRouter sends job listing, guidance & one CV to LLM

OpenRouter

Job listing (listingxx.txt)

Job listing guidance (three-form)

One CV (electrated text)

LLM

API key stays local

5 Pydantic validates each evaluation, then rank all candidates

Pydantic

Schema validation Type safety

Required fields (Score ranges)

2 1 3

Deterministic ranking of all candidates

6 Generate timestamps (output folder with bundle files

/output/ 2025-05-28, 143022

selected candidates/ (output)

scoring, sheet.md (Markdown

interview questions.md (Markdown

Stop on unreadable CV, failed request, or invalid response — no partial completed bundle.

When the Process Stabilizes, Productize It



Use agents to discover the workflow; use your own app to run the repeatable path

Write our own app when

- The workflow is stable and repeatable
- Inputs and outputs are known
- The same controls run every time
- Volume, cost, auditability matter
- Predictable failure handling matters

VS

Use agentic tools when

- The problem is still being explored
- Requirements change frequently
- Exceptions dominate
- Fast prototyping is essential
- Broad research is more important than repetability

Stable process → deterministic application. Unstable process → agenic exploration.

Why a Dedicated App Scales Better

The process is encoded once, then reused consistently

Agentic rediscovery



More turns.
More tokens.
Unpredictable results.

VS.

Reusable application workflow

Deterministic, validated, and reusable

Input folders → Extract text from PDFs → OpenRouter + CV to LLM → Validate & rank → Timemarks & outputs

Encoded process = consistent results at scale

Token efficiency: send only the listing, scoring guidance, and one CV when needed; do not resend broad context or tool instructions.

Throughout: determinate orchestration can process candidates in a predictable sequence and parallelize safely later.

Predictable cost: fewer exploratory turns and less repeated context reduce token usage and make spend easier to estimate.

Reliability and auditability: validation, ranking, fail-fast cleanup, and output structure are enforced by code rather than inferred each run.

The Practical Hybrid: Agents for Change, Apps for Scale

Keep the boundary flexible while the process evolves, then lock in the proven path



1 Explore

using agentic tools to test prompts, edge cases, and candidate criteria

- Test prompts
- Probe edge cases
- Refine candidate criteria

2 Observe

capture successful steps, inputs, outputs, and failure modes

- Log successful steps
- Record inputs & outputs
- Capture failure modes

3 Codify

move the stable workflow into Python services

- Pydantic validation
- pypdf extraction
- Deterministic ranking
- File-based outputs

4 Operate

run the app repeatedly, monitor errors and exceptions, and return unusual cases to agentic review

- Monitor runs
- Track errors & exceptions
- Escalate unusual cases to agentic review

The LLM remains inside a controlled boundary and never receives API keys or direct filesystem access.

Try now

- Start a new folder on your machine (/hr_screening)
- Use seedprompt_build.md to write the HR_build.md
- Copy and paste the HR_build.md and HR_dummy data
- Prompt to start the implementation
- To run automated test, prompt -
    - Start a browser. Test the process end to end. Patch error when it shows up.

Sample

HR Candidate Screening

Local screening workspace

Candidate batch

batch_01

Job listing

listing_01

Validated 10 readable PDF candidates.

Top candidates

3

OpenRouter API key

Model handle

nvidia/nemotron-3-ultra-550b-a55b:free

Review inputs

Generate interview bundle

To pull my project

- Ask agent to visit
  - https://github.com/dennistamyf/vibe_class_hr_demo
- Start a new project folder
  - Prompt:
  - visit https://github.com/dennistamyf/vibe_class_hr_demo
  - pull project from this repo. set it up, then start the dashboard. update necessary file paths.